

Thiết kế ASIC kích thước nhỏ cho thuật toán mã khối Magma trên công nghệ Skywater 130nm

Đào Bá Anh, Nguyễn Ngọc Quỳnh, Đinh Tiến Thành

Học viện Kỹ thuật Mật mã, Hà Nội, Việt Nam

Email: {daobaanh, quynhnn, thanhdt}@actvn.edu.vn

Tóm tắt nội dung—Ngày nay, diện tích phần cứng và mức điện năng tiêu thụ của vi mạch sử dụng trong các thiết bị nhúng, hạng nhẹ, có yêu cầu bảo mật thường bị giới hạn chặt chẽ. Bài báo này đề xuất và trình bày một thiết kế ASIC có kích thước nhỏ, thực thi thuật toán mã khối Magma, một trong hai thuật toán mã khối được định nghĩa trong tiêu chuẩn GOST R 34.12-2015 của Liên bang Nga, dành cho các thiết bị có tài nguyên hạn chế. Thiết kế này được viết bằng Verilog, được tổng hợp, mô phỏng và kiểm tra trên FPGA Xilinx Kintex-7 XC7K160T, và tổng hợp trên ASIC với công nghệ Skywater 130nm sử dụng bộ công cụ EDA mã nguồn mở OpenLANE2. Khi triển khai trên FPGA, thiết kế RTL thuật toán mã khối Magma có khả năng hoạt động ở tần số tối đa 251,32 MHz với thông lượng mã hóa/giải mã là 487,4 Mbps, sử dụng 273 Look-up table (LUTs) và 138 flip-flop (FFs), chiếm lần lượt 0,27% và 0,07% số LUT và FF có sẵn trên FPGA Xilinx Kintex-7 XC7K160T. Khi tổng hợp trên ASIC với công nghệ Skywater 130nm, thiết kế này đạt tần số hoạt động 50 MHz với diện tích phần cứng là 0,048mm².

Index Terms—Magma, triển khai trên FPGA, OpenLANE2, Skywater 130nm, Mã khối, GOST

I. GIỚI THIỆU

Các thuật toán mã khối (block cipher) là những nguyên thủy mật mã cơ bản và đóng vai trò là thành phần cốt lõi trong nhiều giao thức mật mã khác nhau. Chúng được sử dụng rộng rãi để bảo mật dữ liệu trong truyền thông và lưu trữ. Mã khối có thể được định nghĩa như những thuật toán tất định, hoạt động trên các khối dữ liệu đầu vào có độ dài cố định để tạo ra các khối đầu ra có cùng độ dài, dựa trên một khóa bí mật có độ dài cố định. Một thuật toán mã khối bao gồm thuật toán mã hóa và hàm nghịch đảo của nó, được gọi là thuật toán giải mã. Cả hai đều sử dụng cùng một khóa bí mật làm đầu vào. Thuật toán mã hóa thực hiện biến đổi bản rõ (plaintext) thành bản mã (ciphertext). Quá trình biến đổi này được điều khiển bởi giá trị của khóa bí mật. Ngược lại, thuật toán giải mã cần có cùng khóa bí mật đó để khôi phục chính xác bản rõ từ bản mã đầu vào.

Trong nhiều năm qua, nhiều thuật toán mã khối đã được chuẩn hóa và ứng dụng rộng rãi trong các hệ thống nhúng dân sự. Một số thuật toán mã khối nổi tiếng có thể kể đến là Advanced Encryption Standard (AES), Data Encryption Standard (DES), và phiên bản nâng cao của nó là Triple DES (3DES). Thuật toán mã khối đối xứng được sử dụng rộng rãi nhất. AES được Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) chuẩn hóa vào năm 2001 [1]. AES hoạt động trên các khối dữ liệu 128 bit và hỗ trợ độ dài khóa 128, 192 hoặc 256 bit. Nó sử dụng cấu trúc mạng hoán vị – thay thế (SPN:

Substitution-Permutation Network), được đánh giá cao nhờ biên an toàn mạnh và khả năng triển khai hiệu quả cả trong phần mềm lẫn phần cứng. Trước AES, DES đã được chuẩn hóa vào những năm 1970 [2], cụ thể là vào năm 1977. Đây là một trong những thuật toán mã khối đầu tiên được triển khai rộng rãi. Tuy nhiên, DES chỉ sử dụng khóa có độ dài 56 bit và kích thước khối 64 bit. Kích thước khóa nhỏ này đã khiến DES dễ bị tấn công vét cạn (brute-force). Triple DES (3DES) được giới thiệu như một giải pháp tăng cường bằng cách áp dụng DES ba lần với nhiều khóa để cải thiện mức độ an toàn [3] vào năm 1999. Dù vậy, 3DES hiện cũng đã bị loại bỏ do kém hiệu quả và xuất hiện nhiều lỗ hổng bảo mật mới.

Năm 2015, Cơ quan An ninh Liên bang Nga (FSB) đã công bố Tiêu chuẩn Quốc gia Liên bang Nga GOST R 34.12-2015. Đây là tiêu chuẩn mã khối đối xứng dùng để mã hóa dữ liệu. Tiêu chuẩn GOST R 34.12-2015 chuẩn hóa hai thuật toán mã khối đối xứng, bao gồm: Magma, một phiên bản cải tiến của thuật toán GOST 28147-89 với kích thước khối 64 bit và độ dài khóa 256 bit, và Kuznyechik, một thuật toán mới hơn có kích thước khối 128 bit và độ dài khóa 256 bit [4]. Thuật toán Kuznyechik sử dụng cấu trúc SPN tương tự như AES và được thiết kế nhằm hướng đến các ứng dụng yêu cầu mức độ an toàn, bảo mật cao. Trong khi đó thuật toán Magma sử dụng cấu trúc Feistel 32 vòng, sử dụng khóa có độ dài 256-bit và thực thi trên các khối dữ liệu độ dài 64 bit. Do đó thuật toán Magma ít phức tạp hơn, phù hợp với các thiết bị nhỏ gọn, có yêu cầu bảo mật vừa phải. Tuy Magma là một thuật toán mã khối đã được chuẩn hóa theo tiêu chuẩn GOST R 34.12-2015 của Liên bang Nga, nhưng thuật toán này không được phổ biến rộng rãi như các thuật toán mã khối khác như AES, DES hay 3DES. Do phạm vi sử dụng toàn cầu hạn chế và mức độ phổ biến thấp, số lượng công bố học thuật tập trung vào hiện thực phần cứng của Magma tương đối ít, đặc biệt trên các nền tảng như FPGA hoặc ASIC.

Trong nghiên cứu này, chúng tôi trình bày một thiết kế RTL cho thuật toán mã khối Magma. Thiết kế RTL được điều khiển bởi một máy trạng thái hữu hạn (FSM), reset không đồng bộ chủ động ở mức cao, giao tiếp qua giao diện bus theo Giao thức Wishbone [5]. Thiết kế này có thể hoạt động ở tần số tối đa 251,32 MHz, đạt thông lượng cao 487,4 Mbps trong khi chỉ sử dụng một lượng tài nguyên FPGA rất nhỏ, bao gồm 273 Slice LUTs và 138 Slice Registers khi triển khai trên Xilinx FPGA Kintex-7. Khi tổng hợp bằng bộ công cụ EDA mã nguồn mở OpenLANE2 với công nghệ Skywater 130nm, thiết kế này đạt tần số hoạt động 50 MHz với diện tích phần cứng là 0.048mm².

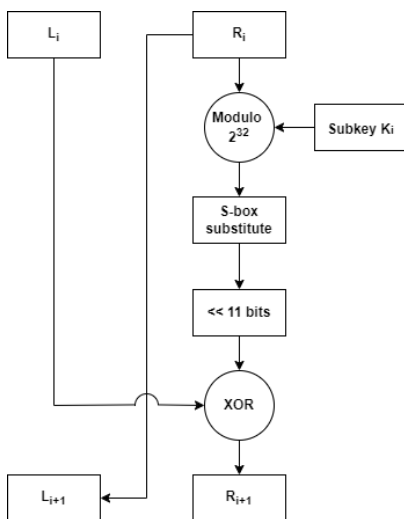
Diện tích này tương đương với ba khối diện tích ô tiêu chuẩn của dự án chế tạo chip đa dự án (Multi Project Wafer) Tiny Tapeout với chi phí chỉ khoảng 200 Euro [6].

Bố cục của bài báo được trình bày như sau: Mục II mô tả ngắn gọn thuật toán mã khối Magma. Mục III phân tích chi tiết thiết kế RTL khối thực thi thuật toán Magma được đề xuất. Mục IV đưa ra các kết quả thực tế khi triển khai thiết kế RTL đề xuất trên FPGA Xilinx Kintex-7 và trên ASIC với công nghệ Skywater 130nm. Cuối cùng, mục V đánh giá kết quả đạt được và đề xuất hướng phát triển kết quả trong tương lai.

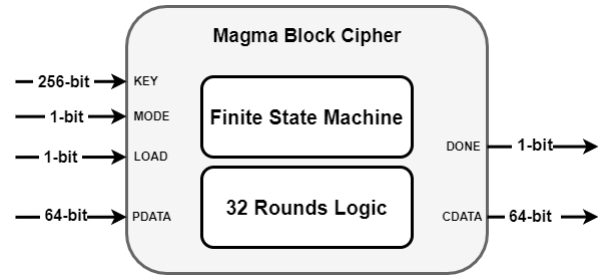
II. THUẬT TOÁN MÃ KHỐI MAGMA

Magma là một thuật toán mã khối sử dụng khóa bí mật đối xứng 256-bit để mã hóa hoặc giải mã các khối dữ liệu có độ dài 64 bit. Thuật toán Magma áp dụng cấu trúc mạng Feistel 32 vòng lặp. Trong quá trình mã hóa, khối dữ liệu bản rõ 64-bit đầu vào được chia thành hai nửa, L_i và R_i và đưa qua các hàm biến đổi vòng (round functions). Mỗi biến đổi hàm vòng bao gồm một hàm thay thế (substitution) và một hàm hoán vị (permutation). Ở mỗi vòng, nửa phải R_i được cộng theo modulo-2³² với khóa con của vòng hiện tại K_i . Kết quả của phép cộng modulo-2³² này sau đó được chia thành tám đoạn 4-bit và được đưa qua giai đoạn thay thế bằng các hộp S 4-bit (S-boxes). Các S-box này là những bảng tra cứu (look-up table) được định nghĩa sẵn, được thiết kế để đảm bảo phép thay thế phi tuyến tính. Tiếp theo, giá trị sau khi thay thế được dịch trái theo bit (bit-wise left shift) 11 bit. Cuối cùng, nửa trái L_i được XOR với giá trị đã dịch để tạo ra nửa phải của vòng tiếp theo R_{i+1} . Nửa trái của vòng tiếp theo L_{i+1} chính là nửa phải của vòng hiện tại, tức R_i .

Hàm vòng của Magma được minh họa trong Hình 1. Quá trình giải mã của thuật toán mã khối Magma về cơ bản giống với quá trình mã hóa, chỉ khác là thực hiện theo thứ tự ngược lại. Các khóa con được sử dụng trong các vòng giải mã chính là các khóa con của vòng mã hóa, nhưng cũng được áp dụng theo thứ tự đảo ngược.



Hình 1. Hàm biến đổi vòng của thuật toán Magma.



Hình 2. Module Magma được đề xuất khi gán với bus hệ thống.

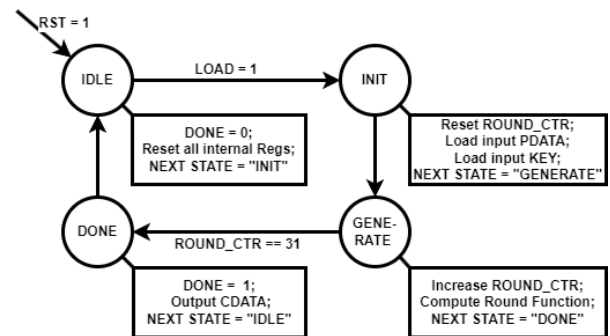
III. THIẾT KẾ RTL CỦA KHỐI THỰC THI THUẬT TOÁN MAGMA

Hình 2 mô tả đề xuất thiết kế RTL của khối thực thi thuật toán Magma, với các thanh ghi điều khiển (control registers) để sẵn sàng tích hợp vào hệ thống SoC dựa trên giao thức Wishbone.

Thiết kế RTL của khối thực thi thuật toán Magma có bốn cổng đầu vào, bao gồm: KEY 256-bit cho khóa bí mật 256-bit, MODE 1-bit để lựa chọn chế độ mã hóa hoặc giải mã, PDATA 64-bit cho khối dữ liệu đầu vào 64-bit, và LOAD 1-bit để bắt đầu quá trình mã hóa/giải mã. Thiết kế RTL của khối thực thi thuật toán Magma có hai cổng đầu ra, bao gồm: CDATA 64-bit cho khối dữ liệu đầu ra 64-bit, và DONE 1-bit để báo hiệu rằng quá trình mã hóa/giải mã đã hoàn tất. Ngoài ra, thiết kế khối RTL này còn có tín hiệu đầu vào Clock và tín hiệu Reset mức cao hoạt động (active high Reset). Tuy nhiên, các tín hiệu này được lược bỏ trong Hình 2 để đơn giản hóa mô tả.

Hoạt động của module RTL thực thi thuật toán Magma được điều khiển bởi một máy trạng thái hữu hạn (Finite State Machine – FSM). Cơ chế này được mô tả trong Hình 3. FSM này có bốn trạng thái như sau:

Trước hết, sau khi tín hiệu Reset được kích hoạt, module RTL thực thi thuật toán Magma sẽ chuyển vào trạng thái IDLE. Ở trạng thái này, tất cả các thanh ghi nội bộ và tín hiệu đầu ra DONE đều được đặt về 0. Đồng thời, khóa bí mật 256-bit và khối dữ liệu đầu vào 64-bit được nạp lần lượt vào các cổng KEY và PDATA. Cổng MODE cũng cần được cấu hình ở mức logic thấp cho chế độ mã hóa, hoặc mức logic-cao cho chế độ giải mã.



Hình 3. Máy trạng thái hữu hạn của module Magma.

Tại thời điểm khi xảy ra chuyển mức từ 0 sang 1 tại cổng đầu vào LOAD, FSM sẽ chuyển trạng thái từ IDLE sang INIT. Trong trạng thái INIT, các giá trị KEY và PDATA được gán vào các thanh ghi nội bộ, đồng thời bộ đếm vòng (round counter) được đặt lại về vòng 0. Sau khi việc nạp dữ liệu và đặt lại bộ đếm hoàn tất, FSM chuyển trạng thái từ INIT sang GENERATE.

Trong trạng thái GENERATE, FSM điều khiển lõi logic thực hiện hàm vòng như đã mô tả trong Mục II. Trong giai đoạn này, bộ đếm vòng đếm từ 0 đến 31 để đảm bảo toàn bộ 32 vòng mã hóa/giải mã được thực hiện. Khi bộ đếm vòng đạt giá trị 31, điều đó có nghĩa là toàn bộ quá trình mật mã đã hoàn tất. FSM khi đó chuyển trạng thái từ GENERATE sang DONE.

Trong trạng thái DONE, cổng đầu ra DONE được đặt lên mức logic-cao, và dữ liệu đã mã hóa hoặc giải mã sẽ xuất hiện tại cổng đầu ra CDATA. Sau đó, FSM chuyển trạng thái từ DONE trở lại IDLE, sẵn sàng cho quá trình mã hóa hoặc giải mã tiếp theo.

IV. KẾT QUẢ TRIỂN KHAI

A. Kết quả triển khai trên FPGA:

Module RTL thực thi thuật toán Magma được triển khai trên bo mạch FPGA SakuraX. FPGA chính của bo mạch này là chip Xilinx Kintex-7 XC7K160T. Kết quả sử dụng tài nguyên sau khi triển khai (post-implementation resource utilization) trên thiết bị XC7K160T của thiết kế module RTL thực thi thuật toán Magma được trình bày trong Bảng I. Cụ thể, thiết kế RTL đã đề xuất cần sử dụng 273 Slice LUTs và 138 Slice Registers, tương đương với 0.27% và 0.07% tổng số Slice LUTs và Slice Registers khả dụng trên thiết bị FPGA Xilinx Kintex-7 XC7K160T.

Bảng I
TÀI NGUYÊN SỬ DỤNG CỦA MODULE MAGMA

Tài nguyên	Sẵn có	sử dụng	Tỷ lệ
LUT	101400	273	0,27
FF	202800	103	0,07
BRAM	0	0	0
DSP	0	0	0

Tần số hoạt động tối đa của thiết kế module Magma được trích xuất từ báo cáo thời gian sau khi triển khai (post-implementation timing report) của công cụ Vivado 2020.2 với ràng buộc tần số cho tín hiệu clock là 100 MHz. Số chu kỳ xung nhịp tối thiểu có thể được tính toán bằng phép tính sau

$$T_{min} = Constraints\ Period - WNS = 3,979\ ns \quad (1)$$

$$f_{max} = \frac{1}{T_{min}} = 251,319\ MHz \quad (2)$$

B. Kết quả triển khai với OpenLANE:

OpenLANE2 là một bộ công cụ EDA (Electronic Design Automation) mã nguồn mở, được phát triển nhằm hỗ trợ quy trình tổng hợp, bố trí – định tuyến và tạo layout hoàn chỉnh cho các thiết kế số dựa trên công nghệ CMOS. Hệ thống này được xây dựng trên nền tảng các công cụ mã nguồn mở như Yosys, OpenROAD, Magic, Netgen và KLayout, cho phép thực hiện

toàn bộ quy trình từ mã RTL (Register Transfer Level) đến file GDSII sẵn sàng cho chế tạo. Ngoài ra, OpenLane hỗ trợ nhiều công nghệ quy trình khác nhau, trong đó nổi bật là SkyWater 130nm PDK (Process Design Kit). Sử dụng OpenLANE2, thiết kế RTL khối thực thi thuật toán mã khối Magma được triển khai trên công nghệ ASIC, quy trình Skywater 130nm. Công cụ OpenLANE2 cho phép người dùng làm việc theo các quy trình cung cấp sẵn. Trong đó, quy trình SynthesisExploration cho phép thử nhiều chiến lược tổng hợp khác nhau nhằm xác định chiến lược nào tối ưu hơn — bằng cách giảm thiểu diện tích hoặc tối đa hóa độ trễ dương (slack), từ đó tăng tần số hoạt động của thiết kế ASIC. Kết quả chạy quy trình SynthesisExploration đối với thiết kế RTL khối thực thi thuật toán mã khối Magma được thể hiện trong Hình 4. Kết quả này được tính toán với cấu hình ràng buộc chu kỳ clock đầu vào là 40ns.

SYNTH_STRATEGY	Gates	Area (μm^2)	Worst Setup Slack (ns)	Total -ve Setup Slack (ns)
AREA 0	2223	27259.894400	23.534646897442...	0.0
AREA 1	2215	27209.846400	23.534646897442...	0.0
AREA 2	2217	27304.937600	23.955810457834...	0.0
AREA 3	3250	32223.404800	23.939091386789...	0.0
DELAY 0	2539	31378.844800	24.287188059240...	0.0
DELAY 1	2503	30668.163200	20.588628273895...	0.0
DELAY 2	2519	30740.732800	22.525217884050...	0.0
DELAY 3	2597	31792.992000	23.011449397309...	0.0
DELAY 4	2991	32622.537600	24.273842289928...	0.0

Hình 4. Kết quả quy trình SynthesisExploration

Kết quả chạy quy trình SynthesisExploration cho thấy khi sử dụng chiến lược tối ưu AREA 1, ước tính diện tích thiết kế sẽ đạt nhỏ nhất đạt khoảng 27210 μm^2 . Trong khi đó, giá trị Worst Setup Slack dương thể hiện phần dư thừa về thời gian trong mỗi chu kỳ clock, với giá trị lớn nhất là 24,29 40ns.

Dựa trên kết quả ước tính từ quy trình SynthesisExploration, thiết kế RTL được đưa vào quy trình đầy đủ gồm các bước tổng hợp, bố trí, đi dây để tạo ra file GDS cuối cùng với các cấu hình như được thể hiện trong Hình 5, các giá trị khác giữ mức mặc định được cung cấp trong tài liệu [7]. cần lưu ý rằng kết quả quy trình SynthesisExploration chỉ mang tính định hướng. Thực tế khi bố trí, đi dây cần phải tính thêm diện tích cho các thành phần như chân IO của khối macro nên cần tăng diện tích cho phép bố trí các cell và đường dây. Việc này cũng dẫn đến cần tăng chu kỳ clock tối thiểu cho thiết kế để đảm bảo file GDS cuối cùng vượt qua được tất cả các bài kiểm tra signoff. Các bài kiểm tra signoff bao gồm: kiểm tra Layout-versus-Schematic, kiểm tra Design Rules Check, kiểm tra Static Timing Analysis, và kiểm tra Antenna.

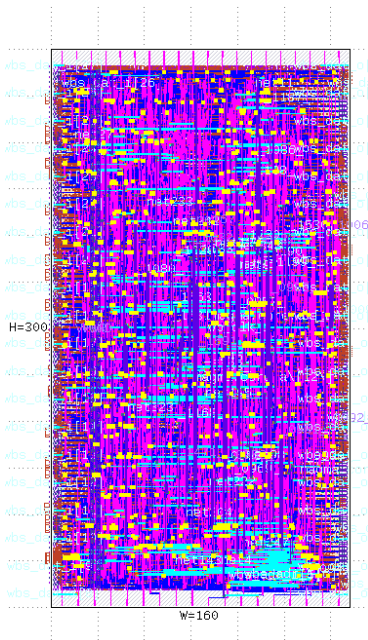
Kết quả bố cục ASIC cuối cùng của thiết kế module Magma được hiển thị ở hình 6. Bố cục này được tạo ra với các cấu hình OpenLANE2: phương thức tổng hợp AREA 1, tần số hoạt động 50MHz, kích thước mạch là 0,048mm². Bố cục này đều vượt qua tất cả các bài kiểm tra signoff của bộ công cụ OpenLANE2 và đặc biệt có kích thước nhỏ, sẵn sàng có thể chế tạo theo chương trình Tintapeout với mức chi phí chỉ khoảng 200 Euro.

V. KẾT LUẬN

Nghiên cứu này đề xuất một triển khai phần cứng của mã khối Magma trên ASIC, công nghệ Skywater130nm. Triển khai đề xuất yêu cầu 273 Slice Look-up Tables (LUTs) và 138 Slice

```
"DESIGN_NAME": "magma_wb_wrapper",
"FP_PDN_MULTILAYER": false,
"CLOCK_PORT": "wb_clk_i",
"CLOCK_PERIOD": 20,
"VERILOG_FILES": [
  "/home/daobaanh/caravel_magma_accele",
  "/home/daobaanh/caravel_magma_accele",
  "/home/daobaanh/caravel_magma_accele"
],
"FP_CORE_UTIL": 50,
"GRT_ANTENNA_ITEPS": 18,
"RUN_HEURISTIC_DIODE_INSERTION": true,
"HEURISTIC_ANTENNA_THRESHOLD": 200,
"MAX_TRANSITION_CONSTRAINT": 1.5,
"DESIGN_REPAIR_MAX_SLEW_PCT": 40,
"DESIGN_REPAIR_MAX_CAP_PCT": 40,
"DESIGN_REPAIR_MAX_WIRE_LENGTH": 800,
"DEFAULT_CORNER": "max_ss_100C_1v60",
"RUN_POST_GRT_DESIGN_REPAIR": true,
"SYNTH_STRATEGY": "AREA_1",
"PNR_SDC_FILE": "dir::pnr.sdc",
"SIGNOFF_SDC_FILE": "dir::signoff.sdc",
"FP_PIN_ORDER_CFG": "dir::pin_cfg.cfg",
"FP_SIZING": "absolute",
"DIE_AREA": [0, 0, 160, 300]
```

Hình 5. Các giá trị tùy chỉnh trong thiết kế



Hình 6. Kết quả bố cục của khối Magma

Flip-flops (FFs), tương ứng với 0,27% và 0,07% số LUTs và FFs có sẵn trong thiết bị Xilinx Kintex-7 XC7K160T. Triển khai của chúng tôi đạt được những kết quả đáng kể; các kết quả thể hiện yêu cầu tài nguyên cần sử dụng thấp, tần số tối đa có thể thực thi là 251,32 MHz, và thông lượng tối đa là 487,4 Mbps.

Với kết quả triển khai sử dụng công cụ EDA OpenLANE2, thiết kế ASIC của khối thực thi thuật toán Magma có diện tích 0,048mm² với tần số hoạt động 50MHz. Kết quả triển khai có thể được chế tạo thực tế với chi phí thấp, phục vụ thuận lợi cho công việc đào tạo về thiết kế vi mạch số.

TÀI LIỆU

- [1] National Institute of Standards and Technology (NIST), "FIPS PUB 197: Advanced Encryption Standard (AES)," <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>, 2001.
- [2] —, "FIPS PUB 46: Data Encryption Standard (DES)," <https://nvlpubs.nist.gov/nistpubs/Legacy/FIPS/NIST.FIPS.46.pdf>, 1977.
- [3] —, "FIPS PUB 46-3: Data Encryption Standard (DES)," <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.46-3.pdf>, 1999.

- [4] Federal Agency on Technical Regulating and Metrology, "GOST R 34.12-2015: Information Technology. Cryptographic Data Security. Block Cipher. General Specifications," https://www.tc26.ru/en/standard/gost/GOST_R_3412-2015.pdf, 2015, standard published in Moscow, Russia.
- [5] M. Sharma and D. Kumar, "Wishbone bus architecture-a survey and comparison," *arXiv preprint arXiv:1205.1860*, 2012.
- [6] M. Venn and U. Shaked, "Tiny tapeout: Quicker, easier and cheaper to make your own chip," 2025, accessed: 2025-10-07. [Online]. Available: <https://tinytapeout.com/>
- [7] Efabless, "Openlane 2 documentation," 2025, accessed: 2025-10-08. [Online]. Available: <https://openlane2.readthedocs.io/en/latest/index.htm>